

Fast Plans, Faithful Actions: Closing the Planning-Execution Gap in Hierarchical Vision-Language-Action Models

Chuanliang Xie^{1,*}, Boyu Ma^{1,*}, Gen Li¹, Yizhou Liu¹, Houwang Chen¹, Xinyu Zhou¹, Jianfei Yang^{1,†}

¹MARS Lab, Nanyang Technological University

*Equal contribution, †Corresponding authors

Hierarchical vision-language-action (VLA) systems consist of a high-level vision-language planner and a low-level action expert that generates continuous actions. This hierarchical design has practical value only if the planner can generate plans fast enough to meet real-time control requirements, and the resulting plans actually contribute to the generation of action. We study one such system, a waypoint hierarchy pipeline adapted from $\pi_{0.5}$, and find that neither requirement is satisfied. This baseline relies on token-level autoregressive decoding (Token-AR) to generate a waypoint plan, requiring 57 very expensive vision-language model (VLM) forward passes. However, we find that erasing the waypoint endpoints has little effect on task success. Two findings reveal the misalignment of planner-executor: the planner generates outputs at an excessively fine granularity, and the executor underuses plans as a control condition. We address the latency issue with waypoint-aligned block-autoregressive decoding (Block-AR), and plan underuse issue with normalized goal modulation (NGM), a layer-wise goal path constrained by phase gating and anti-shortcut training so that the waypoint influences action generation maintaining other signals. Our method reduces the maximum number of VLM forward passes from 57 to 8 on LIBERO, including one prefix prefill, and achieves an $8.7\times$ reduction in planning latency on a Rokae dual-arm robot. With normalized goal modulation and anti-shortcut training, Block-AR’s success rate on LIBERO-Long increases from 91.0% to 96.2%, while its average success rate across the four suites increases from 95.85% to 98.45%. On three bimanual tasks with this robot, success rates remain comparable across methods.

Correspondence: jianfei.yang@ntu.edu.sg, chuanlia001@e.ntu.edu.sg



1 Introduction

Vision-language-action (VLA) models generate robot actions directly from image and language inputs (Kim et al., 2025; Black et al., 2025; Physical Intelligence et al., 2025). In recent years, more and more works have adopted a hierarchical structure to realize this mapping: a high-level vision-language planner first generates a plan in an intermediate form, such as a trajectory sketch, a set of keyposes, or a waypoint sequence, and then an execution stage converts this plan into continuous actions (Li et al., 2025; Hwang et al., 2026; Liu et al., 2025; Huang et al., 2025; Wu et al., 2026). The benefit of this division of labor is that a large vision-language model (VLM) can fully exploit its advantages in semantic and spatial reasoning, and can incorporate action-space chain-of-thought (Wu et al., 2026), while the execution stage handles continuous control.

However, this division of labor places two requirements on the interface between the planner and the executor. Requirement 1 (R1): the plan must be generated in time. When the VLM generates the plan token by token in an autoregressive manner, the cost of decode computation grows sharply with the number of output tokens, which poses a great challenge to the real-time performance of the whole system. Requirement 2 (R2): the plan must genuinely contribute to behavior. Besides the plan, the low-level policy can usually also see the key/value (K/V) representations of the images and language; in this case, empirical risk minimization does not force it to determine its control behavior through this intermediate representation. Prior work has analyzed failures in which a learned executor deviates from the path predicted by the planner (Li et al., 2025); this paper goes one step further and directly measures this interface through controlled interventions. Fig. 1

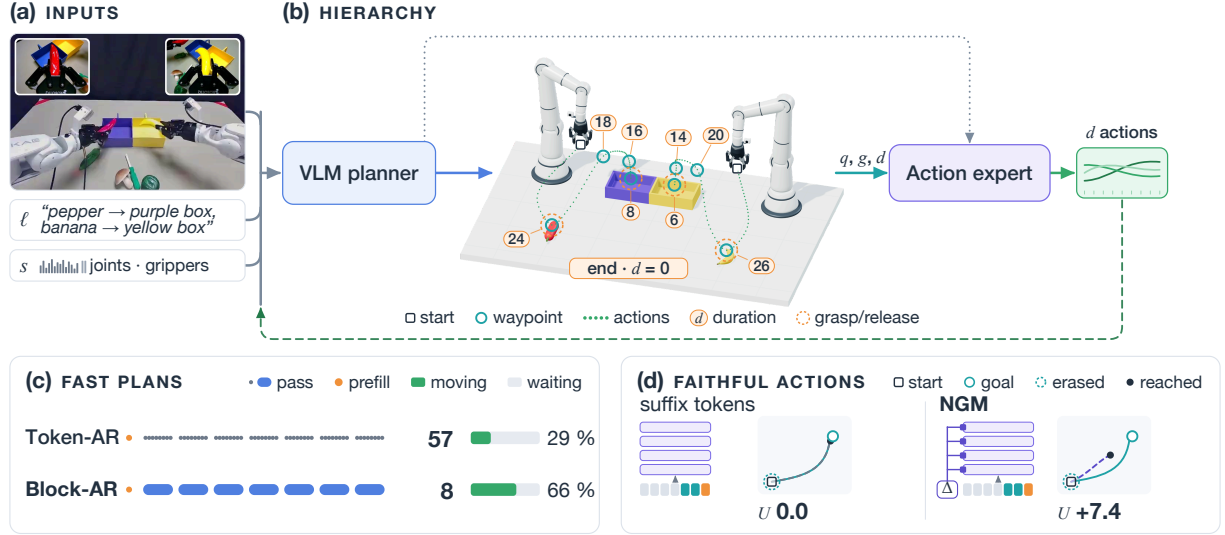


Figure 1 Fast plans, faithful actions. (a) Inputs (Rokae AR5-5 dual arm, pepper–banana placement): external and wrist images, instruction ℓ , state s . (b) A PaliGemma planner (Gemma-2B language model) predicts (q, g, d) waypoints (target configuration, gripper, duration). The schematic shows four waypoints per arm (hollow rings; orange dashed rings: grasp/release), illustrative per-arm step counts (linked pills), and corresponding action beads. Deployed dual-arm blocks share one d ; $d = 0$ ends the plan. A 300M flow-matching expert, which also attends to the planner’s image and language prefix (dotted), maps each waypoint to d actions; the robot executes, observes, and replans (dashed loop). (c) For one seven-waypoint LIBERO plan, Token-AR needs one prefix prefill plus one backbone pass per value token (dots), 57 passes in total; Block-AR needs the same prefill plus one pass per waypoint (pills), 8 in total. On the robot, plan latency falls from 1094 to 125 ms and the arms move 66% instead of 29% of the time (bars). (d) Endpoint-erasure test on LIBERO-Long: with images, instruction, state, and flow noise fixed, the commanded endpoint is set to the segment start. Erasure does not reduce success with suffix-token conditioning ($U = 0$), but reduces success by 7.4 points with normalized goal modulation (NGM), which injects the goal into every expert layer.

outlines the hierarchical architecture we study and the two requirements above.

To study these two requirements, we adapt $\pi_{0.5}$ into a hierarchical architecture, which we call Waypoint Token-AR (Fig. 1(a), (b)). In this architecture, the PaliGemma backbone (Beyer et al., 2024) serves as the planner and is responsible for predicting waypoints in the robot’s configuration space; each waypoint consists of three parts: a target configuration, a gripper state, and an execution duration. The action expert trained with flow matching (Lipman et al., 2023) receives the current state and the waypoint through suffix tokens; we call this baseline route suffix-token conditioning. It should be noted that this baseline is built by us specifically for controlled experiments and is not the original architecture of $\pi_{0.5}$.

We find that neither requirement is satisfied on this baseline. Consider R1 first: generating a LIBERO (Liu et al., 2023) plan containing seven waypoints requires the 2B-parameter language backbone to perform 57 serial backbone passes, including one prefix prefill. On a real dual-arm robot, the time spent generating one plan even exceeds the execution time of the action segment that the plan specifies; as a result, the arms are in a waiting state for 71% of the actual running time. Now consider R2: with the images, language, and flow-matching noise all held fixed, we erase the target endpoint in the plan, and the resulting change in actions is only 0.6% of the change caused by re-sampling the noise once. Erasing the target endpoint during online execution has little effect on task success. By contrast, the action expert does show a measurable response to the execution duration.

These two findings demonstrate a misalignment between the planner and the executor. The executor consumes a plan one waypoint at a time, but the planner generates it one token at a time. The waypoint-aligned block-autoregressive decoding (Block-AR) removes this granularity mismatch: the planner generates a complete waypoint in each VLM forward pass while the causal dependency between waypoints can be preserved (Fig. 1(c)). The executor takes in the waypoint as a condition, but the waypoint contributes little to action

generation. To solve this issue, we design normalized goal modulation (NGM), which gives the goal a layer-wise path into the action expert (Fig. 1(d)). We explain why an unconstrained path produces a “displacement-integrator shortcut” and then constrain it with phase gating and anti-shortcut training, thereby ensuring that waypoint information contributes to action generation while preserving the model’s responses to other input signals.

The main contributions of this paper are as follows:

- We conduct a controlled study on a waypoint hierarchy built on $\pi_{0.5}$. By measuring action sensitivity and task utility under waypoint interventions, we find that standard suffix-token conditioning leaves the spatial target endpoint almost ineffective.
- We propose waypoint-aligned block-autoregressive planning. It reduces the maximum serial planning depth from 57 forward passes to 8, and in experiments reduces the generation latency per plan from 1094 ms to 125 ms, raises the proportion of motion time from 29% to 66%, and supports a learnable end-of-plan marker.
- We identify the displacement-integrator shortcut that makes direct goal injection fail, and then design normalized goal modulation (NGM) as a layer-wise goal path whose phase gate, condition noise, and dropout mitigate the training shortcut. NGM improves Block-AR performance on all four LIBERO suites while giving the waypoints a real effect on actions.

2 Related Work

2.1 Waypoint and keypose interfaces

Sparse intermediate representations have a long research history in imitation learning, including generating motion between keyposes with diffusion or consistency models (Xian et al., 2023; Ma et al., 2024; Yu et al., 2026), salient-point hybrid methods (Sundaresan et al., 2025), and Automatic Waypoint Extraction (AWE). AWE selects a minimal waypoint set under the constraint of a piecewise-linear reconstruction-error threshold (Shi et al., 2023). We extract waypoints offline with AWE under gripper-transition and fixed-horizon constraints (Section 4.1). The usual design of hierarchical VLAs is to let the VLM generate the sparse plan itself, for example: 2-D trajectory sketches (Li et al., 2025), pixel-depth waypoint sequences (Hwang et al., 2026), camera-frame 3-D waypoints executed by a frozen action expert (Liu et al., 2025), image-space trajectory tokens executed via splines (Huang et al., 2025), and coarse-to-fine control (Wu et al., 2026). Our VLM also writes waypoints, each with a duration and an end flag that give the low-level policy timing and termination information. We study the serial cost of the above hierarchy and the influence of waypoints (Table 1).

2.2 Decoding efficiency in VLAs

OpenVLA generates discretized robot actions autoregressively (Kim et al., 2025), whereas π_0 uses a flow-matching action expert to generate continuous action chunks (Black et al., 2025). Serial VLM decoding can limit the control frequency of autoregressive VLAs. Approaches to improving efficiency include frequency-domain action compression in FAST (Pertsch et al., 2025), parallel decoding and consistency decoding (Song et al., 2025b,a), asynchronous reasoning and execution in FiS-VLA (Chen et al., 2025), and joint discrete/continuous training with knowledge insulation for fast action-expert inference (Driess et al., 2025). Block decoding itself is not a new concept. The difference is that we align each block with one semantically meaningful waypoint, and jointly decode its target configuration, gripper state, duration, and $d = 0$ termination flag in a single backbone forward pass; this reduces serial depth and improves termination decoding (Section 6.2).

2.3 Conditions ignored by the policy

Learned policies can rely on shortcuts or underuse relevant conditions. Related failures include causal confusion and copycat behavior in behavior cloning (de Haan et al., 2019; Wen et al., 2020), and weak language conditioning in VLAs (Lian et al., 2026; Glossop et al., 2026). Guidance can strengthen condition adherence in generative models (Ho and Salimans, 2022; Zheng et al., 2023). Hindsight experience replay, in turn, relabels

Table 1 Planner-to-executor designs in hierarchical VLAs.

Method	Plan space	Executor	Serial cost grows with	Interface assessment
HAMSTER (Li et al., 2025)	2-D image path	separate policy	path points	failure attribution
3D HAMSTER (Hwang et al., 2026)	pixel + depth path	separate flow policy	path points	guidance ablation
GAE (Liu et al., 2025)	camera-frame 3-D points	frozen point-cloud expert	waypoints	goal-noise ablation
NoTVLA (Huang et al., 2025)	pixel + depth keyposes	spline + replanning	keyposes	deterministic execution
Coarse-to-Control (Wu et al., 2026)	coarse action blocks	same model, AR tokens	blocks + actions	qualitative diagnostics
Ours	robot configuration + duration	flow expert, shared prefix	value slots (Token-AR) / waypoints (Block-AR)	S, U

Serial cost summarizes the decoding structure, not measured latency across methods. Interface assessment describes the reported evaluation or execution mechanism. 3D HAMSTER compares no, 2-D, and 3-D guidance; GAE varies goal-pose noise during training; HAMSTER attributes failures to trajectory deviation; Coarse-to-Control visualizes attention and decoded plans. Our interventions change the waypoint while holding other inputs fixed.

past experience with goals that were actually achieved (Andrychowicz et al., 2017). We encountered a similar problem in our hierarchical architecture: the waypoints generated by the VLM are drowned out by the image and language signals, so that they have almost no influence on the final action generation. To address this problem, we propose closed-loop measurement methods and a corresponding training fix.

3 Problem Formulation

Plan representation. Let o be the current observation (a first-person image and a wrist-camera image), ℓ the language instruction, and $s \in \mathbb{R}^n$ the robot proprioceptive state. The planner π_H predicts at most M waypoint blocks at a time. The executable plan consists of the K waypoints before an end code appears or the decoding limit is reached:

$$P = \pi_H(o, \ell, s) = (w_1, \dots, w_K), \quad w_i = (q_i, g_i, d_i), \quad (1)$$

where $K \leq M$; q_i is the discretized target robot configuration; $g_i \in \{\text{open}, \text{closed}\}^{n_g}$ contains one gripper command for each arm; and $d_i \in \{1, \dots, D\}$ is the number of continuous actions required to reach q_i . A block with $d = 0$ ends the decoded plan; task completion is assessed separately.

Execution. The executor π_L takes each waypoint in turn as the current target state and generates continuous actions accordingly. Given the current state s_i and the waypoint w_i , the executor outputs d_i actions:

$$a_{1:d_i} = \pi_L(o, \ell, s_i, q_i, g_i, d_i; \epsilon), \quad (2)$$

where ϵ is the noise of the generative action expert. When every decoded duration does not exceed the prediction horizon D of the action expert, we call the representation horizon-compatible. This property only guarantees that the requested action segment fits within the output horizon of the action expert; it does not imply that the motion is collision-free or dynamically feasible. Section 4.1 guarantees horizon compatibility by construction.

Faithfulness. Even if a waypoint is horizon-compatible, it may still have almost no effect on actual execution. Because the executor also receives image and language signals through K/V representations, the task information carried by q_i may be ignored or drowned out, especially when each training context corresponds to only one endpoint. We assess the model’s dependence on waypoints with two controlled intervention experiments. When evaluating sensitivity, we fix o, ℓ, s_i , and ϵ and change only the waypoint. Sensitivity compares the action change caused by a waypoint intervention δ with the change caused by re-sampling the action expert’s own noise:

$$S(\delta) = \frac{\text{med } \|\pi_L(\cdot, q + \delta; \epsilon) - \pi_L(\cdot, q; \epsilon)\|}{\text{med } \|\pi_L(\cdot, q; \epsilon') - \pi_L(\cdot, q; \epsilon)\|}, \quad (3)$$

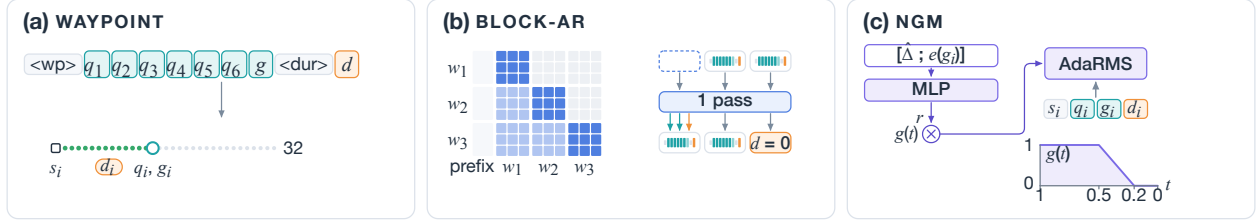


Figure 2 Method. (a) Waypoint representation: a LIBERO waypoint is the token group $\langle \text{wp} \rangle q_1 \dots q_6 g \langle \text{dur} \rangle d$ with $d \in \{1, \dots, 32\}$ (338 reserved token IDs; a robot waypoint has 19 tokens). For block i the expert denoises $D = 32$ actions from the measured state s_i and executes the first d_i (green); the next block starts from the measured state. (b) Block-AR: attention is bidirectional within a block (dark) and causal across blocks (light); inputs are shifted by one block (the first is a learned query, dashed) and targets are not, so one backbone pass yields a whole waypoint through three LM-head calls (configuration, gripper, duration). Decoding produces at most $M = 7$ blocks and stops early when a block predicts $d = 0$. (c) NGM: besides the suffix tokens s_i, q_i, g_i, d_i , the normalized goal $[\hat{\Delta}; e(g_i)]$ with $\hat{\Delta} = (q_i - s_i)/\sigma_{\Delta}$ enters an MLP with a zero-initialized output projection. Its residual r , gated by $g(t)$, is added to the AdaRMS condition $\phi(t) + g(t)r$ of every layer; the gate is open for $t \geq 0.5$ and closed for $t \leq 0.2$. Training noises the endpoint seen by both routes ($\sigma_c = 0.7$) and replaces both by null embeddings with probability 0.15; inference uses the clean endpoint and the gate only.

where the norm is first computed as the root mean square (RMS) over the executed time steps and valid action dimensions, and then the median is taken over all segments. Utility is the paired change in task success rate after the spatial plan is erased at test time, i.e., $U = \text{SR}(P) - \text{SR}(\tilde{P})$, where $\tilde{q}_i := s_i$. Thus, $U > 0$ indicates that erasing the endpoint hurts task success. Neither of the above measures alone can prove that the conditioning input is effective; we interpret them together with the success rate under the intact plan and the model’s sensitivity to visual input. Section 6.3 explains why this joint interpretation is necessary: a model can have a very large S and still fail to complete the task.

4 Method

Our system (Fig. 2) is derived from $\pi_{0.5}$ (Physical Intelligence et al., 2025): a PaliGemma backbone (Beyer et al., 2024) (consisting of a SigLIP encoder and a Gemma-2B language model) serves as the planner, and a separate 300M-parameter Gemma action expert inherited from $\pi_{0.5}$ serves as the executor and is trained with flow matching (Lipman et al., 2023). The action expert accesses the cached vision and language-instruction prefix through cross-attention, and the two modules are adapted with low-rank adaptation (LoRA) (Hu et al., 2022) within the same coupled model. We add an interface for outputting waypoints to the planner and compare two different ways in which the action expert generates continuous actions conditioned on the waypoints. Success rates and planning costs are reported in Tables 2 and 3, respectively.

4.1 Waypoint representation and execution

Extraction. We extract waypoints from each demonstration using the dynamic programming algorithm of AWE (Shi et al., 2023). The algorithm uses a segment-error threshold η to select a sparse set of frames. We force the frame at every gripper state transition to be a keyframe and impose a maximum segment-length constraint inside the dynamic program, so that every executable gap falls within $[1, D]$, where $D = 32$ is the number of control steps in the action expert’s prediction horizon. For the real robot running at 30 Hz, this corresponds to at most 1.07 s; for the LIBERO simulator running at 20 Hz, it corresponds to at most 1.6 s. The number of steps from the segment start to the target waypoint is set as the positive duration of that waypoint, and a separate terminal block with $d = 0$ is added. Fig. 3 shows an example; segment statistics are in Table 3.

Direct interpolation and inverse kinematics alone do not provide scene-conditioned corrections between waypoints. Following the motivation for a perception-conditioned executor in GAE (Liu et al., 2025), we use the flow-matching action expert to generate continuous actions between waypoints.

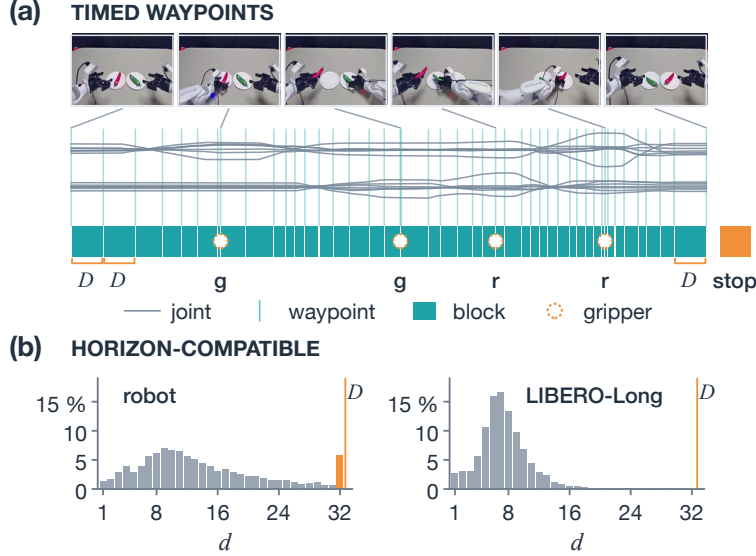


Figure 3 Waypoints extracted from demonstrations. (a) Pepper-swap episode 2: 634 steps (21 s) become 45 waypoints. The top row shows the start, four gripper transitions, and end. The curves show the 14 joint angles (left arm above, right arm below; centered with a shared scale); vertical cuts mark waypoints, and the blocks below encode segments (q, g, d) with width proportional to duration d . Brackets identify the three segments capped at $D = 32$ steps; rings mark gripper transitions (g = grasp, r = release; left, right, right, left arm). The orange $d = 0$ block marks the end of the plan. (b) Segment-duration distributions on a shared scale: robot (6,158 training segments; median 12, 5.8% at the cap) and LIBERO-Long (13,815 segments; median 7, longest 29).

Table 2 Success rate (%) on the four LIBERO suites.

Method	Spatial	Object	Goal	Long	Avg.	Δ vs $\pi_{0.5}$	Δ vs Token-AR
$\pi_{0.5}$ (released, as reported)	98.8	98.2	98.0	92.4	96.85	—	—
Ours, Token-AR + suffix cond.	98.0	99.0	97.6	92.2	96.70	−0.15	—
Ours, Block-AR + suffix cond.	98.6	97.6	96.2	91.0	95.85	−1.00	−0.85
Ours, Block-AR + NGM	99.2	99.4	99.0	96.2	98.45	+1.60	+1.75

$N = 500$ per suite (10 tasks $\times$ 50 initial states). The $\pi_{0.5}$ row lists the values reported in the official OpenPI LIBERO evaluation README (Physical Intelligence, 2025); the three waypoint variants use the common protocol of Section 5 and share data, initialization, and training steps. Δ : difference of the four-suite averages in points.

Tokenization. As shown in Fig. 2(a), we reuse 338 token IDs at the tail of the existing PaliGemma vocabulary to represent waypoints: 300 token IDs shared by the six continuous configuration dimensions (each dimension is normalized separately before use), two gripper tokens, 34 duration codes, and two structural tokens ($\langle \text{wp} \rangle$, $\langle \text{dur} \rangle$). The duration tokens use $d = 0$ to indicate end-of-plan and $d \in \{1, \dots, 32\}$ to indicate actual durations. Code 33 marks the current-state input and may also arise when training shifts the input observation by ± 1 step, adjusting the first duration while keeping waypoint targets fixed. Any decoded 33 is clamped to 32. A LIBERO waypoint is represented by the token group $\langle \text{wp} \rangle q_1 \dots q_6 g \langle \text{dur} \rangle d$; on the dual-arm robot, the token group contains 14 joint slots and two gripper slots. Decoding is slot-constrained: the logits of each slot are masked to its corresponding token family, so every decoded plan is syntactically valid. The planner prefix contains the instruction, the discretized current state, and the camera images (two in LIBERO; one external-view image and one wrist-view image on the real robot). Decoding produces at most $M = 7$ blocks; if a terminal block is produced, it is also counted.

Execution. The suffix-conditioned action expert takes the segment start state, the target (q_i, g_i) , and the duration d_i as suffix tokens, and denoises an action chunk of length D , of which the first d_i actions are executed. The next segment starts from the measured state. In simulation, a new plan is requested only after the current plan has been fully executed; on the real robot, a new plan is requested as soon as the first segment has been executed. The real robot uses receding-horizon replanning because its observation changes

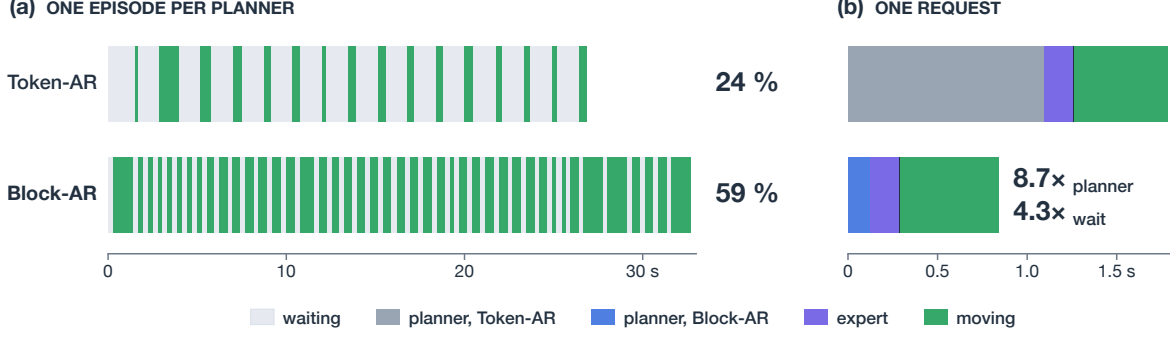


Figure 4 Robot planning latency and motion duty cycle. (a) Recorded shelf-placement timelines for Token-AR and Block-AR on a common time axis. Waiting comprises planner, expert, and transport; motion denotes commanded execution at 30 Hz. Motion occupies 24% and 59% here, versus 29% and 66% over 35 telemetry episodes (Table 3). Both episodes were stopped before placement. (b) Mean request and commanded-motion durations (497 Token-AR / 936 Block-AR requests). Planner latency falls from 1094 to 125 ms (8.7 $\times$); expert latency remains near 160 ms. Transport is the hairline segment. The request round trip drops from 1266 to 293 ms (4.3 $\times$), below the roughly 0.5 s of commanded motion.

faster than a full plan can be executed. The code $d = 0$ indicates the end of the current decoded plan; whether the task terminates is determined by the simulator or by a human operator.

4.2 Waypoint-aligned block-autoregressive planning

Compact Token-AR must resolve the value slots sequentially: at the maximum $M = 7$, one prefix prefill plus 56 value-token forward passes requires a total of 57 expensive serial backbone forward passes. This becomes the most time-consuming part of the system. To speed up planner inference, we treat each waypoint as one block. Attention remains causal across blocks and is bidirectional within a block; the inputs are shifted by one block rather than one token, so block k is predicted from the preceding blocks $< k$. The first block is predicted from a learnable query with a zero-initialized residual. The labels are not shifted, and both planners use the same training samples and target values. At inference, each decoding forward pass generates all semantic value slots of one waypoint; decoding stops when a block predicts $d = 0$. The three token families in a block are read out through three language-model-head calls. Therefore, a maximum-length LIBERO plan needs only 8 forward passes (one prefill plus seven waypoint forward passes) instead of 57. Because the number of Block-AR forward passes is independent of the number of slots per waypoint, the more degrees of freedom the robot has, the more pronounced the speed-up: $57 \rightarrow 8$ in LIBERO and $120 \rightarrow 8$ on the robot with 16 degrees of freedom. Fig. 4 illustrates the resulting latency savings.

4.3 Normalized goal modulation

The suffix-conditioned expert receives the goal only through the suffix tokens in (2). Because it can also see the images and language, nothing in the training loss forces it to route task information through the waypoint. We add an additional, more direct route for the goal. We write the goal as a displacement relative to the current state and scale it per dimension: $\hat{\Delta} = (q_i - s_i) / \sigma_{\Delta}$, where $\sigma_{\Delta, j} = \max(1.4826 \text{ MAD}_j, 0.25 \text{ std}_j, 10^{-3})$ is computed over the normalized demonstration displacements; it is then concatenated with a learnable gripper embedding $e(g_i)$, and finally mapped to a residual r by an MLP whose output projection is zero-initialized. r is added to the AdaRMS condition of every layer of the expert, so that the goal can reach every layer of the network, thereby strengthening the influence of the waypoint on action generation. At initialization $r = 0$, and the expert behaves the same as before, so the original weights are not disrupted. Table 4 compares the conditioning variants.

If no constraint is placed on this route (which we call naive goal injection), it over-amplifies the influence of the waypoint. In the demonstrations, the relative goal is not an independent condition: $q_i - s_i$ approximately equals the integral of the very actions the expert is learning to produce, so delivering it to every layer amounts

Table 3 Planning cost (A) and waypoint statistics (B).

A. Planning cost			B. Waypoint statistics and end-of-plan code		
	Token-AR	Block-AR		LIBERO-Long	robot
Serial passes / plan, max (LIBERO / robot)	57 / 120	8 / 8	Segment length p50 / p95 (steps)	7 / 13	12 / 32
Plan latency, robot (ms)	1094	125	Segments at the cap $D = 32$	0%	5.8%
Request round trip, robot (ms)	1266	293		Token-AR	Block-AR
Motion duty cycle, robot	28.7%	65.6%	End-of-plan recall / precision	0.00 / –	0.61 / 0.85
			Decoded plans with end marker, robot	0 / 497	109 / 936

A: one prefix prefill and at most $M = 7$ decoded blocks. Robot timings are means over the 497 Token-AR / 936 Block-AR requests of 35 telemetry episodes; expert and transport add about 160 and 7 ms per request. Duty cycle: fraction of wall-clock time executing actions, averaged over episodes. B: 13,815 LIBERO-Long and 6,158 robot segments after extraction. End-of-plan recall/precision: 2,172 held-out robot windows, 270 true endings; Block-AR has 164 hits and 30 false alarms. The last row counts online plans containing an end marker. No LIBERO evaluation decoded a duration above D .

to leaking the label into the condition. The expert can then fit the training loss by integrating the displacement, without having to look at the images. We call this the displacement-integrator shortcut; Section 6.3 shows that it yields a very large S but a low success rate.

NGM keeps the deep route and constrains it to discourage this shortcut. The goal is useful for the rough direction of a segment, whereas the final refinement should come from what the expert sees, so we use a fixed phase gate to restrict the deep route to the coarse stage of denoising:

$$\text{cond}_\ell = \phi(t) + g(t)r, \quad g(t) = \text{clip}\left(\frac{t - 0.2}{0.3}, 0, 1\right). \quad (4)$$

The shortcut also requires the condition to correspond exactly to the label, so during training we add Gaussian noise to the endpoint ($\sigma_c = 0.7$ in the normalized $\hat{\Delta}$ domain) while the action supervision remains unchanged; in this way the goal can only serve as an approximate target, and the expert must cross-check it against the images. Finally, with probability 0.15 we replace the goal with a learnable null embedding, so that the expert remains capable without a goal. Both perturbations are applied to the suffix tokens and the deep route simultaneously; otherwise, an unperturbed suffix-token endpoint would bypass the regularization and bring the label in unchanged. At inference, noise and dropout are turned off while the gate remains on; the reported results use the ordinary conditional branch and do not use classifier-free guidance (Ho and Salimans, 2022).

4.4 Training

The three waypoint variants use the same data, initialization (the released $\pi_{0.5}$ weights), random seed, and number of training steps. The backbone receives $\nabla\mathcal{L}_{\text{CE}} + \lambda\nabla\mathcal{L}_{\text{FM}}$, where $\lambda = 1.7$. Knowledge insulation blocks action-expert gradients from reaching the backbone (Driess et al., 2025); our variant retains this gradient path with weight $\lambda = 1.7$. We apply LoRA (Hu et al., 2022) (rank 16) to all linear layers in the backbone, the vision encoder, and the action expert, giving 46.4M–49.6M trainable parameters, or 1.27–1.35% of the 3.66B total. The global batch for training the LIBERO models contains 80 planner windows and 80 action-expert segments.

5 Experimental Setup

Simulation. We conduct experiments on the four LIBERO suites (Liu et al., 2023) (Spatial, Object, Goal, and Long). We train one model on the union of the four suites (1,693 episodes); waypoints are extracted with $\eta = 0.008$ in the six-dimensional end-effector configuration, and waypoint selection also takes the open/close state of the gripper into account. For evaluation, each suite contains 10 tasks, and each task is evaluated on the 50 official initial states, i.e., $N = 500$ episodes per suite; we follow the OpenPI LIBERO evaluation protocol (Physical Intelligence, 2025), with step budgets of 220/280/300/520 steps, respectively, plus 10 settling steps. All evaluations start from the measured state, and LoRA weights are evaluated without exponential moving average (EMA).

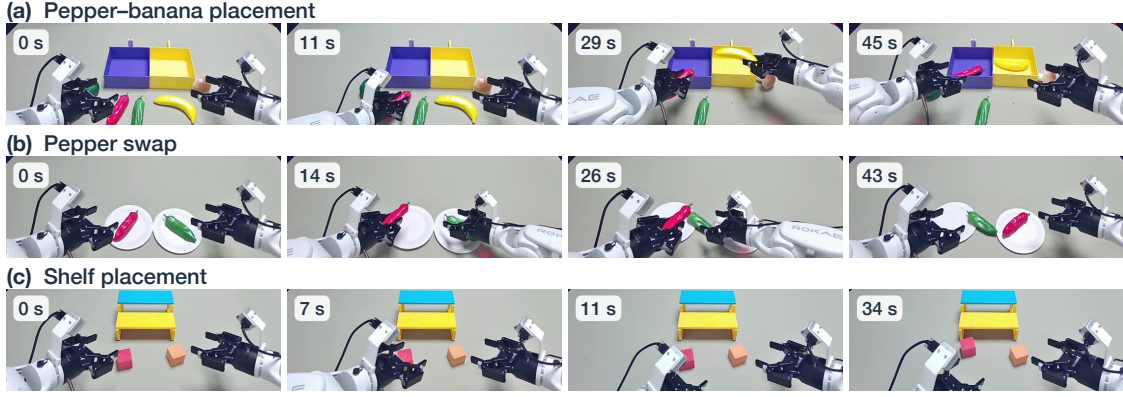


Figure 5 Representative dual-arm sequences from Block-AR telemetry rollouts (external camera, seconds since start); success rates are reported separately in Table 5. (a) Pepper-banana placement. (b) Pepper swap. (c) Shelf placement, stopped at 34 s with the red block carried to the shelf but not placed.

Table 4 Waypoint dependence and visual responsiveness on LIBERO-Long.

System	B	S_{endpoint} (%)	S_{duration} (%)	S_{image}	Retention	$\text{SR}_{\text{intact}}$ (%)	U (pp)
Block-AR + suffix	0.118	0.6	19	0.330	100%	91.0	0.0
Token-AR + naive	0.074	348	106	0.090	27%	15	+14
Block-AR + NGM	0.105	42.6	26.4	0.27	82%	96.2	+7.4

All variants use the shared four-suite training and evaluation protocol. Naive injection adds the deep goal route without gate, condition noise, or dropout. Each intervention changes one input while holding the others and the flow noise fixed; S and B use the same 64 frozen segments. B : RMS action change under re-sampling the model’s own flow noise, the denominator of S . S : median action change under endpoint erasure or duration +4 steps, relative to B . S_{image} : absolute RMS change under a main-camera swap; retention: its ratio to the suffix-token row. $\text{SR}_{\text{intact}}$: success with the unmodified plan; U : its drop after endpoint erasure on the same $N = 500$ initial states per model.

Real robot. The dual-arm platform consists of a pair of Rokae AR5-5 arms with Robotiq 2F-85 grippers; each arm is equipped with a wrist camera, and there is one additional external camera. We set up three bimanual tasks: placing a pepper and a banana into two boxes in the presence of distractors; swapping a red and a green pepper between two plates; and color-matched shelf placement (Fig. 5). We collected 154 demonstrations in total (51, 51, and 52 for the three tasks); after excluding four episodes with inconsistent video length and holding out 15 for validation and 15 for testing, the remaining 120 (38, 40, and 42 for the three tasks) are used for training. Waypoints are extracted on the 14 joints with $\eta = 0.015$ rad. Each method is run for 20 trials per task in alternating order, with success judged by an operator.

6 Results

6.1 End-to-end performance

Table 2 compares the results of the systems on the LIBERO dataset. Waypoint Token-AR differs from the reported $\pi_{0.5}$ results by at most 0.8 points on each suite (-0.15 points on average). Replacing Token-AR with Block-AR changes success by between $+0.6$ points (Spatial) and -1.4 points (Object, Goal), -0.85 points on average relative to Token-AR, while the number of forward passes of the VLM planner per plan drops from 57 to 8. With NGM, the same Block-AR system improves on every suite, by 2.6 points on average and by 5.2 points on LIBERO-Long. For any suite, one point corresponds to 5 of its 500 evaluation episodes; in addition, each variant was trained only once.

6.2 Fast: the cost of a plan

Table 3 and Fig. 4 quantify requirement (R1). On the real robot, each waypoint carries 19 tokens; Token-AR takes 1094 ms to generate a plan and 1266 ms for the full request including the action expert, longer than

Table 5 Real-robot results.

Task	Token-AR	Block-AR	+NGM	Plan (ms)	Duty (%)
Pepper-banana	17/20	16/20	18/20	1144/129	31/62
Pepper swap	16/20	15/20	17/20	1055/123	30/68
Shelf placement	15/20	15/20	16/20	1085/122	23/65
All	48/60	46/60	51/60	1094/125	29/66

Successes over 20 trials per task and method, judged by the operator. Plan latency (ms, Token-AR / Block-AR) is averaged over requests and motion duty cycle (%) over episodes, from the 35 telemetry rollouts of Fig. 4. The All row averages over all requests and episodes, not over the three task rows.

the roughly 0.5s action segment it commands, so the arms are in motion only 28.7% of the time. Block-AR shortens the planning time to 125 ms ($8.7\times$) and the round-trip latency to 293 ms, and raises the motion duty cycle to 65.6% (per task in Table 5). The remaining latency comes mainly from the action expert (160 ms per segment) rather than the planner. Among 2,172 held-out robot planning windows with 270 true plan endings, Token-AR never emits $d = 0$, whereas Block-AR achieves 61% recall and 85% precision.

6.3 Faithful: does the executor follow the plan?

Diagnosis. Table 4 quantifies the extent to which the method with suffix-token conditioning satisfies requirement R2. On the same 64 action segments, with all other inputs and the flow-matching noise held fixed, the change in actions caused by erasing the planned endpoint amounts to only 0.6% of the change caused by re-sampling the noise. However, the action expert is not generally insensitive to all of its inputs: swapping the main camera image changes the actions by 281% of the noise re-sampling baseline; swapping the language instruction changes them by 158%; and increasing the waypoint duration by 4 control steps changes them by 19% of the noise baseline. In closed-loop evaluation, erasing every spatial endpoint in the plan causes no drop in task success, i.e., $U = 0.0$.

Sensitivity alone is insufficient to show that conditioning is effective. Naive goal injection raises endpoint sensitivity to 348% and duration sensitivity to 106%. However, the model’s response to visual changes falls to only 27% of the suffix-token-conditioned baseline, and task success drops sharply to 15%. All of its failures are timeouts, and its behavior shows the following pattern: the model relies excessively on the goal displacement proposed by the planner and does not make sufficient use of visual information for online correction, which is the displacement-integrator shortcut of Section 4.3. Endpoint sensitivity must therefore be read together with the success rate under the intact plan, the task utility of the endpoint, and the retained response to other inputs.

Normalized goal modulation. With NGM in place of suffix-only conditioning, the success rate of the Block-AR system on LIBERO-Long in closed-loop evaluation rises from 91.0% to 96.2%. The endpoint sensitivity of the NGM model increases from 0.6% to 42.6%, and duration sensitivity from 19% to 26.4%; at the same time, the sensitivity of the action expert to image changes remains at 82%. Endpoint erasure, which costs nothing under suffix-token conditioning, now reduces success by 7.4 points: the expert uses the waypoint while keeping most of its response to vision.

6.4 Real robot

Table 5 and Fig. 5 present the setup and results of the dual-arm robot experiments. Telemetry provides the latency and motion duty cycle: on every task, Block-AR reduces per-plan latency to 1/8.6–1/8.9 of the original and more than doubles the fraction of time the arms are in motion. Each method is run for 60 trials; Token-AR, Block-AR, and Block-AR + NGM succeed 48, 46, and 51 times, respectively. On real-robot evaluation data not used in training, Block-AR also predicts the next waypoint more accurately: on 2,172 held-out planning windows, with the first-waypoint joint-angle error averaged over both arms, Token-AR has an error of 0.156 rad and Block-AR 0.112 rad, a 28% reduction for the latter.

7 Conclusion

Our experiments show that end-to-end task success alone establishes neither planning efficiency nor effective use of the planner–executor interface in hierarchical VLAs. In a controlled Waypoint Token-AR hierarchy derived from $\pi_{0.5}$, we identify two forms of planner–executor misalignment: planning has a high inference latency, and the planned spatial endpoint barely contributes to action generation. Waypoint-aligned Block-AR keeps success within 1.4 points of Token-AR on every LIBERO suite while reducing serial planner passes from 57 to 8. NGM improves the Block-AR system on all four suites, by 2.6 points on average, and makes the executor depend more strongly on the planned endpoint: endpoint sensitivity rises from 0.6% to 42.6%, and erasing the endpoint reduces success by 7.4 points.

Scope and limitations. Block-AR and NGM are designed for and validated on one hierarchy derived from $\pi_{0.5}$: Block-AR assumes fixed-format plan token groups, and NGM relies on the AdaRMS conditioning of the $\pi_{0.5}$ expert. Whether they transfer to other VLA backbones is untested. Each model variant was trained only once, so we cannot estimate the variability across training runs. The real-robot study covers three tasks with 20 trials per task and method; this sample size limits the precision of success-rate estimates and conclusions about small between-method differences.

References

- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Adv. Neural Inf. Process. Syst.*, volume 30, 2017. https://papers.nips.cc/paper_files/paper/2017/hash/453fadb8a1a3af50a9df4df899537b5-Abstract.html.
- Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. PaliGemma: A versatile 3B VLM for transfer, 2024. <https://arxiv.org/abs/2407.07726>. arXiv:2407.07726.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. In *Proc. Robot. Sci. Syst. (RSS)*, 2025. doi: 10.15607/RSS.2025.XXI.010. <https://www.roboticsproceedings.org/rss21/p010.html>.
- Hao Chen, Jiaming Liu, Chenyang Gu, Zhuoyang Liu, Renrui Zhang, Xiaoqi Li, Xiao He, Yandong Guo, Chi-Wing Fu, Shanghang Zhang, and Pheng-Ann Heng. Fast-in-Slow: A dual-system VLA model unifying fast manipulation within slow reasoning. In *Adv. Neural Inf. Process. Syst.*, volume 38, pages 98049–98083, 2025. doi: 10.52202/085713-3276. https://proceedings.neurips.cc/paper_files/paper/2025/hash/8cf3760422b9d4505589a97c8f9569e7-Abstract-Conference.html.
- Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *Adv. Neural Inf. Process. Syst.*, volume 32, 2019. https://papers.nips.cc/paper_files/paper/2019/hash/947018640bf36a2bb609d3557a285329-Abstract.html.
- Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and Sergey Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. In *Adv. Neural Inf. Process. Syst.*, volume 38, pages 102867–102888, 2025. doi: 10.52202/085713-3439. https://papers.nips.cc/paper_files/paper/2025/hash/94e936034d12bcd04834ec2773f02aff-Abstract-Conference.html.
- Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. CAST: Counterfactual labels improve instruction following in vision-language-action models. *IEEE Robot. Autom. Lett.*, 11(10):11785–11792, Oct. 2026. doi: 10.1109/LRA.2026.3726383. <https://doi.org/10.1109/LRA.2026.3726383>.
- Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, 2022. <https://arxiv.org/abs/2207.12598>. arXiv:2207.12598.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. <https://arxiv.org/abs/2106.09685>.
- Zheng Huang, Mingyu Liu, Xiaoyi Lin, Muzhi Zhu, Canyu Zhao, Zongze Du, Ye Lin, Xiaoman Li, Yiduo Jia, Hao Zhong, Hao Chen, and Chunhua Shen. NoTVLA: Semantics-preserving robot adaptation via narrative action interfaces, 2025. <https://arxiv.org/abs/2510.03895>. arXiv:2510.03895.
- Dongyoon Hwang, Byungkun Lee, Dongjin Kim, Hyojin Jang, Hoiyeong Jin, Jueun Mun, Minho Park, Hojoon Lee, Hyunseung Kim, and Jaegul Choo. 3D HAMSTER: Bridging planning and control in hierarchical vision language action models through 3D trajectory guidance, 2026. <https://arxiv.org/abs/2606.31329>. arXiv:2606.31329.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proc. Conf. Robot Learn. (CoRL)*, volume 270, pages 2679–2713, 2025. <https://proceedings.mlr.press/v270/kim25c.html>.
- Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Caelan Garrett, Fabio Ramos, Dieter Fox, Anqi Li, Abhishek Gupta, and Ankit Goyal. HAMSTER: Hierarchical action models for open-world robot manipulation. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, pages 24040–24068, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/3bfee3bc6639c36e6e7b058db909f760-Abstract-Conference.html.
- Shijie Lian, Bin Yu, Xiaopeng Lin, Laurence T. Yang, Zhaolong Shen, Changti Wu, Yuzhuo Miao, Cong Huang, and Kai Chen. LangForce: Bayesian decomposition of vision language action models via latent action queries. In *Proc. Int. Conf. Mach. Learn. (ICML)*, 2026. <https://arxiv.org/abs/2601.15197>.

- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. <https://arxiv.org/abs/2210.02747>.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Adv. Neural Inf. Process. Syst.*, volume 36, pages 44776–44791, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract-Datasets_and_Benchmarks.html.
- Mingyu Liu, Zheng Huang, Xiaoyi Lin, Muzhi Zhu, Canyu Zhao, Yating Wang, Haoyi Zhu, Hao Chen, and Chunhua Shen. GAE: Unleashing physical potential of VLM with generalizable action expert, 2025. <https://arxiv.org/abs/2510.03896>. arXiv:2510.03896.
- Xiao Ma, Sumit Patidar, Iain Haughton, and Stephen James. Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 18081–18090, 2024. https://openaccess.thecvf.com/content/CVPR2024/html/Ma_Hierarchical_Diffusion_Policy_for_Kinematics-Aware_Multi-Task_Robotic_Manipulation_CVPR_2024_paper.html.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. In *Proc. Robot. Sci. Syst. (RSS)*, 2025. doi: 10.15607/RSS.2025.XXI.012. <https://www.roboticsproceedings.org/rss21/p012.html>.
- Physical Intelligence. LIBERO Benchmark. OpenPI, GitHub, 2025. <https://github.com/Physical-Intelligence/openpi/blob/2d70d966582e711128ad8358d8dbf23d2cc3d658/examples/libero/README.md>. Accessed: Sep. 18, 2026.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization, 2025. <https://arxiv.org/abs/2504.16054>. arXiv:2504.16054.
- Lucy Xiaoyang Shi, Archit Sharma, Tony Z. Zhao, and Chelsea Finn. Waypoint-based imitation learning for robotic manipulation. In *Proc. Conf. Robot Learn. (CoRL)*, volume 229, pages 2195–2209, 2023. <https://proceedings.mlr.press/v229/shi23b.html>.
- Wenxuan Song, Jiayi Chen, Pengxiang Ding, Yuxin Huang, Han Zhao, Donglin Wang, and Haoang Li. CEED-VLA: Consistency vision-language-action model with early-exit decoding, 2025a. <https://arxiv.org/abs/2506.13725>. arXiv:2506.13725.
- Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Zhijun Li, Donglin Wang, Lujia Wang, Jun Ma, and Haoang Li. PD-VLA: Accelerating vision-language-action model integrated with action chunking via parallel decoding. In *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pages 13162–13169, 2025b. doi: 10.1109/IROS60139.2025.11247519. <https://doi.org/10.1109/IROS60139.2025.11247519>.
- Priya Sundaresan, Hengyuan Hu, Quan Vuong, Jeannette Bohg, and Dorsa Sadigh. What’s the move? Hybrid imitation learning via salient points. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, pages 51806–51821, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/8063ef83cf43c341bc124b6175f2729d-Abstract-Conference.html.
- Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories. In *Adv. Neural Inf. Process. Syst.*, volume 33, pages 2564–2575, 2020. https://proceedings.neurips.cc/paper_files/paper/2020/hash/1b113258af3968aaf3969ca67e744ff8-Abstract.html.
- Jinhao Wu, Shiduo Zhang, Yicheng Liu, Xiaopeng Yu, Sixian Li, Siyin Wang, Hang Zhao, Jing Huo, Yang Gao, Jingjing Gong, Xipeng Qiu, and Yu-Gang Jiang. Coarse-to-Control: Action-token planning for vision-language-action models, 2026. <https://arxiv.org/abs/2606.07107>. arXiv:2606.07107.
- Zhou Xian, Nikolaos Gkanatsios, Theophile Gervet, Tsung-Wei Ke, and Katerina Fragkiadaki. ChainedDiffuser: Unifying trajectory diffusion and keypose prediction for robotic manipulation. In *Proc. Conf. Robot Learn. (CoRL)*, volume 229, pages 2323–2339, 2023. <https://proceedings.mlr.press/v229/xian23a.html>.
- Dongjie Yu, Hang Xu, Yizhou Chen, Yi Ren, and Jia Pan. BiKC: Keypose-conditioned consistency policy for bimanual robotic manipulation. In *Algorithmic Foundations of Robotics XVI, Volume 2*, volume 38 of *Springer Proc. Adv. Robot.*, pages 283–302. Springer, Cham, Switzerland, 2026. doi: 10.1007/978-3-032-09970-9_15. https://link.springer.com/chapter/10.1007/978-3-032-09970-9_15.
- Qinqing Zheng, Matt Le, Neta Shaul, Yaron Lipman, Aditya Grover, and Ricky T. Q. Chen. Guided flows for generative modeling and decision making, 2023. <https://arxiv.org/abs/2311.13443>. arXiv:2311.13443.